

Question 1: Find the Second Highest Salary

Problem Statement

You are given an **Employee** table containing employee details, including their salaries.

Write an SQL query to fetch the employee(s) who have the second highest salary.

Table Structure

Sample Table : Employee

EmpID	EmpName	DepartmentID	Salary
101	John	1	90000
102	Alice	2	120000
103	Bob	1	110000
104	David	2	120000
105	Emma	3	95000
106	Chris	1	105000

Expected Output

EmpID	EmpName	Salary
103	Bob	110000

Solution

There are multiple ways to solve this problem:

1. Using **DENSE_RANK()** (Recommended)
2. Using **MAX()**
3. Using a Subquery

4. Using **LIMIT/OFFSET** (Database-specific)

The best approach is to use the **DENSE_RANK()** window function because it correctly handles duplicate salaries.

SQL Query (Using **DENSE_RANK**)

```
WITH SalaryRank AS
(
    SELECT *,
           DENSE_RANK() OVER(ORDER BY Salary DESC) AS SalaryRank
    FROM Employee
)
SELECT EmpID,
       EmpName,
       Salary
FROM SalaryRank
WHERE SalaryRank = 2;
```

Alternative Solution (Using **MAX**)

```
SELECT *
FROM Employee
WHERE Salary =
(
    SELECT MAX(Salary)
    FROM Employee
    WHERE Salary <
    (
        SELECT MAX(Salary)
        FROM Employee
    )
);
```

Tips

- Use **DENSE_RANK()** when duplicate salaries should share the same rank.
- Avoid using **LIMIT** or **TOP** in interview answers unless the interviewer specifies a particular database.
- The **MAX()** approach works well but becomes difficult for the Nth highest salary.

- If two employees earn the second highest salary, both should be returned.
- Mention the time complexity and readability of your chosen approach during interviews.
- Be prepared for follow-up questions such as:
 - Find the 3rd highest salary.
 - Find the Nth highest salary.
 - Find the second highest salary department-wise.
 - Explain the difference between `RANK()`, `DENSE_RANK()`, and `ROW_NUMBER()`.

Note: If multiple employees have the same second highest salary, return all of them.

Question 2: Find the Second Highest Salary in Each Department

Problem Statement

You are given an `Employee` table containing employee details, including their department and salary.

Write an SQL query to fetch the employee(s) who have the second highest salary in each department.

Sample Data: Employee

EmpID	EmpName	DepartmentID	Salary
101	John	10	90000
102	Alice	10	120000
103	Bob	10	110000
104	David	20	100000
105	Emma	20	95000
106	Chris	20	100000
107	Sophia	30	70000
108	James	30	85000
109	Olivia	30	80000

Expected Output

EmpID	EmpName	DepartmentID	Salary
103	Bob	10	110000
105	Emma	20	95000
109	Olivia	30	80000

Solution

There are multiple ways to solve this problem:

1. Using **DENSE_RANK()** (Recommended)
2. Using **RANK()**
3. Using a **Correlated Subquery**
4. Using **MAX()** with Subqueries (More Complex)

The recommended approach is to use the **DENSE_RANK()** window function because it correctly handles duplicate salaries within each department.

SQL Query (Using DENSE_RANK)

```
WITH SalaryRank AS
(
    SELECT
        EmpID, EmpName, DepartmentID, Salary,
        DENSE_RANK() OVER
        (
            PARTITION BY DepartmentID
            ORDER BY Salary DESC
        ) AS SalaryRank
    FROM Employee
)
SELECT
    EmpID,
    EmpName,
    DepartmentID,
    Salary
FROM SalaryRank
```

```
WHERE SalaryRank = 2
ORDER BY DepartmentID, Salary DESC;
```

Alternative Solution (Using Correlated Subquery)

```
SELECT *
FROM Employee e1
WHERE Salary =
(
    SELECT MAX(Salary)
    FROM Employee e2
    WHERE e2.DepartmentID = e1.DepartmentID
    AND e2.Salary <
    (
        SELECT MAX(Salary)
        FROM Employee e3
        WHERE e3.DepartmentID = e1.DepartmentID
    )
)
ORDER BY DepartmentID;
```

Explanation

- **PARTITION BY DepartmentID** divides the data into separate groups for each department.
- **ORDER BY Salary DESC** ranks employees from highest to lowest salary within each department.
- **DENSE_RANK()** assigns the same rank to employees with the same salary.
- Filtering **SalaryRank = 2** returns the second highest salary in every department.

Tips

- Use **PARTITION BY** when ranking data within groups.
 - Prefer **DENSE_RANK()** over **ROW_NUMBER()** when duplicate salaries should receive the same rank.
 - If multiple employees share the second highest salary, all of them should be returned.
 - **ROW_NUMBER()** returns only one employee, whereas **DENSE_RANK()** returns all employees with the same rank.
 - This is one of the most frequently asked SQL interview questions involving **Window Functions**.
-

Question 3: Find the Top-Paid Employee in Each Department

Problem Statement

You are given an **Employee** table containing employee details, including their department and salary.

Write an SQL query to fetch the employee(s) with the highest salary in each department.

Sample Table: Employee

EmpID	EmpName	DepartmentID	Salary
101	John	10	90000
102	Alice	10	120000
103	Bob	10	110000
104	David	20	100000
105	Emma	20	95000
106	Chris	20	100000
107	Sophia	30	70000
108	James	30	85000
109	Olivia	30	80000

Expected Output

EmpID	EmpName	DepartmentID	Salary
102	Alice	10	120000
104	David	20	100000
106	Chris	20	100000
108	James	30	85000

Explanation: Department **20** has two employees with the highest salary (**100000**), so both are returned.

Solution

There are multiple ways to solve this problem:

1. Using **DENSE_RANK()** (Recommended)
2. Using **RANK()**
3. Using a **Correlated Subquery**
4. Using **MAX()** with a **JOIN**

The recommended approach is to use the **DENSE_RANK()** window function because it handles ties correctly and is easy to understand.

SQL Query (Using DENSE_RANK)

```
WITH SalaryRank AS
(
    SELECT
        EmpID,
        EmpName,
        DepartmentID,
        Salary,
        DENSE_RANK() OVER
        (
            PARTITION BY DepartmentID
            ORDER BY Salary DESC
        ) AS SalaryRank
    FROM Employee
)
SELECT
    EmpID,
    EmpName,
    DepartmentID,
    Salary
FROM SalaryRank
WHERE SalaryRank = 1
ORDER BY DepartmentID, Salary DESC;
```


Alternative Solution (Using MAX + JOIN)

```
SELECT e.EmpID,  
       e.EmpName,  
       e.DepartmentID,  
       e.Salary  
FROM Employee e  
JOIN  
(  
    SELECT DepartmentID,  
           MAX(Salary) AS MaxSalary  
    FROM Employee  
    GROUP BY DepartmentID  
) d  
ON e.DepartmentID = d.DepartmentID  
AND e.Salary = d.MaxSalary  
ORDER BY e.DepartmentID;
```

Explanation

- **PARTITION BY DepartmentID** creates separate salary rankings for each department.
- **ORDER BY Salary DESC** ranks employees from highest to lowest salary within each department.
- **DENSE_RANK()** assigns the same rank to employees with equal salaries.
- Filtering **SalaryRank = 1** returns the highest-paid employee(s) from every department.

Tips

- Use **DENSE_RANK()** when multiple employees can share the highest salary.
- If only one employee should be returned per department, use **ROW_NUMBER()** instead.
- **MAX()** with **JOIN** is efficient and commonly used in production SQL.
- Avoid using **LIMIT** or **TOP** unless the interviewer specifies a database platform.
- Always consider edge cases, such as departments with only one employee or multiple employees sharing the highest salary.

Note:

- If multiple employees have the same highest salary within a department, return all of them.
- The result should include the department, employee details, and salary.

Question 4: Top 3 Highest Paid Employees in Each Department

Problem Statement

Write an SQL query to find the **top 3 highest-paid employees in each department** from the **Employee** table.

Sample Table: Employee

EmployeeID	EmployeeName	Department	Salary
101	Amit	IT	95000
102	Neha	IT	88000
103	Rahul	IT	75000
104	Sneha	IT	70000
105	Raj	HR	65000
106	Priya	HR	62000
107	Mohit	HR	58000
108	Anjali	HR	55000
109	Vivek	Finance	92000
110	Pooja	Finance	87000
111	Karan	Finance	81000
112	Meena	Finance	79000

Expected Output

EmployeeName	Department	Salary
Amit	IT	95000
Neha	IT	88000
Rahul	IT	75000
Raj	HR	65000
Priya	HR	62000
Mohit	HR	58000
Vivek	Finance	92000
Pooja	Finance	87000
Karan	Finance	81000

Solution

Use the **ROW_NUMBER()** window function to assign a rank to employees **within each department**, ordered by salary in descending order. Then filter the rows where the rank is less than or equal to **3**.

SQL Query

```
WITH RankedEmployees AS
(
    SELECT
        EmployeeID,
        EmployeeName,
        Department,
        Salary,
        ROW_NUMBER() OVER
        (
            PARTITION BY Department
            ORDER BY Salary DESC
        ) AS RankNo
    FROM Employee
)

SELECT
    EmployeeName,
    Department,
```

```
Salary
FROM RankedEmployees
WHERE RankNo <= 3
ORDER BY Department, Salary DESC;
```

Tips

- Use **PARTITION BY Department** to restart the ranking for each department.
 - **ROW_NUMBER()** always returns unique rankings (1, 2, 3, ...), even if salaries are the same.
 - If employees with the **same salary should share the same rank**, use **DENSE_RANK()** or **RANK()** instead.
-

Note

- **ROW_NUMBER()** returns exactly **3 rows per department**.
- If there are salary ties and you want to include **all employees tied for the 3rd highest salary**, replace **ROW_NUMBER()** with **DENSE_RANK()**:

```
WITH RankedEmployees AS
(
    SELECT
        EmployeeID,
        EmployeeName,
        Department,
        Salary,
        DENSE_RANK() OVER
        (
            PARTITION BY Department
            ORDER BY Salary DESC
        ) AS RankNo
    FROM Employee
)

SELECT
    EmployeeName,
    Department,
    Salary
FROM RankedEmployees
WHERE RankNo <= 3
ORDER BY Department, Salary DESC;
```

Interview Follow-up: Explain the difference between **ROW_NUMBER()**, **RANK()**, and **DENSE_RANK()**, and discuss which one is appropriate depending on whether salary ties should be included.